

ELEC 292

Project Report

Introduction To Data Science
Computer Engineering
Faculty of Engineering and Applied Science
Queen's University

Prepared by Group 42:

Shehraj Mutti, XX, 22wk31@queensu.ca

Rafael Costa, XX, 22xwyk@queensu.ca

Victor Chowdhury, XX, 22fkx1@queensu.ca

04/04/2025

Table of Contents

List of Tables	3
List of Figures	3
1. Data Collection	1
2. Data Storage	1
3. Visualization.....	3
3.1 Time Plots	3
3.2 Overlaid Plots	6
3.3 Metadata Visualizations	9
3.4 Insights Gained	10
4. Preprocessing.....	10
4.1 Noise Reduction	10
4.2 Handling Missing Values	10
4.3 Choosing a Window Size	10
4.4 Effectiveness of Preprocessing	11
5. Feature Extraction & Normalization.....	11
5.1 Feature Selection	12
5.2 Feature Extraction Process	12
5.3 Normalization Technique	13
6. Training & Testing the Classifier	13
6.1 Training Pipeline	13
6.2 Model Training and Evaluation	14
7. Model Development.....	16
Appendix	17
References	17

List of Tables

Table 1: Window size and accuracy correlation, used for determining window size in processing.....	11
Table 2: Breakdown on work distribution in project and report.....	17

List of Figures

Figure 1: Hierarchical structure of the HDF5 file used to store project data.....	1
Figure 2: Raw accelerometer data for Rafael's jumping trial 1.	3
Figure 3: Raw accelerometer data for Rafael's walking trial 1.....	4
Figure 4: Raw accelerometer data for Shehraj's jumping trial 1.	4
Figure 5: Raw accelerometer data for Shehraj's walking trial 1.	5
Figure 6: Raw accelerometer data for Victor's jumping trial 1.....	5
Figure 7: Raw accelerometer data for Victor's walking trial 1.	6
Figure 8: Rafael's absolute acceleration of walking and jumping, overlaid.	7
Figure 9: Shehraj's absolute acceleration of walking and jumping, overlaid.	7
Figure 10: Victor's absolute acceleration of walking and jumping, overlaid.....	8
Figure 11: Distribution data used in training from each group member.....	9
Figure 12: Histogram of the feature (mean) in the x-axis, and their frequencies.....	9
Figure 13: Plots showing the difference between the raw data and changes from preprocessing.....	11
Figure 14: ROC curve plot showing relationship between true positive and false positive rate.....	14
Figure 15: Confusion matrix displaying how many times a label was predicted vs what the label truly was.	15
Figure 16: Training model learning curve as set size increases.....	15
Figure 17: Graphical User Interface displaying different button functionalities.....	16
Figure 18: Result of Display Data button on GUI when provided with walking data, showcasing number of windows predicted as walking compared to jumping	17

1. Data Collection

The data collection process was completed using an application called Phyphox, which utilized built-in smartphone sensors to measure device acceleration while automatically excluding the effect of gravity. To diversify the dataset, each team member completed eight trials: four trials involving walking and four trials involving jumping. The team diversified the data by varying the phone's position during each trial—holding it in-hand, placing it in the right pant pocket, the right jacket pocket, and the back pocket. Each trial lasted 40 seconds, resulting in approximately 5 minutes and 30 seconds of data collected per team member. The collected data was exported directly from Phyphox as .csv files and then emailed to transfer them onto a computer. All .csv files were compiled into a dataset, which was stored and made accessible to each team member via Microsoft Teams.

Careful attention was necessary during the data collection process, as several challenges emerged that could compromise data quality. The primary challenge faced was inconsistency in data due to noise and unintended movements. Factors such as loose clothing caused unexpected shifts in the phone's position, negatively impacting the accuracy of the accelerometer readings. To mitigate this issue, team members ensured minimal unnecessary phone movement and conducted repeat trials whenever inconsistencies were observed. Additionally, variability in stride lengths and jumping patterns among team members introduced potential inconsistencies; however, these differences ultimately benefited the dataset by contributing to its diversity. All hardware utilized during data collection consisted of phones equipped with built-in accelerometers. The software tools employed were Phyphox for data recording and exporting, Microsoft Outlook for file transfer, and Microsoft Teams for dataset storage.

2. Data Storage

After collecting our accelerometer data and converting it to .csv files, we implemented a systematic approach to data organization and storage. Each file was assigned a descriptive path name following the convention "Trial # - Activity, Member Name (40s)" to facilitate efficient data extraction in the processing steps. The team selected the Hierarchical Data Format 5 (HDF5) for storage due to its numerous advantages for scientific data management. Figure 1 illustrates the hierarchical structure we implemented in the HDF5 file.

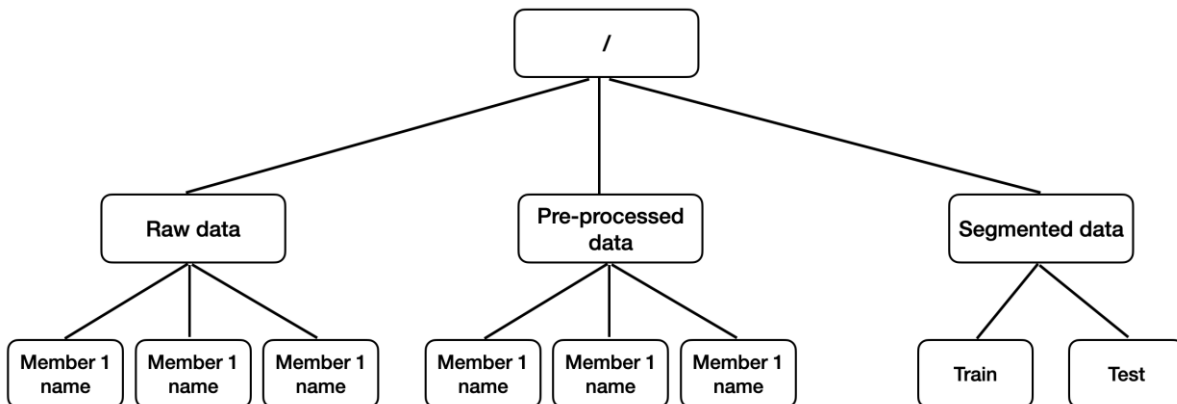


Figure 1: Hierarchical structure of the HDF5 file used to store project data.

The team’s implementation leveraged the h5py Python library to create a three-tiered primary structure. First, the “Raw data” group contains the unmodified accelerometer measurements exactly as collected from the smartphones, preserving the original data integrity for reference. Then, the “Pre-processed data” group stores the cleaned and filtered sensor data after applying preprocessing (detailed in Section 4). Finally, the “Segmented data” group contains the windowed data divided into a “Train” group which holds 90% of the segmented data used for model training and a “Test” group containing the remaining 10% used for model evaluation.

Within both the “Raw data” and “Pre-processed data” groups, the team created member-specific subgroups. Each member's subgroup further contains activity-specific groups (Walking and Jumping), which are then divided into trial-specific groups (Trial 1 through Trial 4). This organization preserves the experimental structure while enabling efficient data access patterns. The following Python code demonstrates our approach to creating this hierarchical structure:

```
1. with h5py.File(output_hdf5, 'w') as hf:
2.     raw_data_group = hf.create_group('Raw data')
3.     preprocessed_data_group = hf.create_group('Pre-processed data')
4.     segmented_data_group = hf.create_group('Segmented data')
5.
6.     train_group = segmented_data_group.create_group('Train')
7.     test_group = segmented_data_group.create_group('Test')
```

For each CSV file, the team utilized the pandas library to load and standardize the data. The team renamed the columns to more intuitive names ('time', 'x', 'y', 'z', and 'magnitude') to simplify processing:

```
1. df_renamed = df.rename(columns={
2.     "Time (s)": "time",
3.     "Linear Acceleration x (m/s^2)": "x",
4.     "Linear Acceleration y (m/s^2)": "y",
5.     "Linear Acceleration z (m/s^2)": "z",
6.     "Absolute acceleration (m/s^2)": "magnitude"
7. })
```

To ensure a good evaluation methodology, we implemented a train-test split using the train_test_split function from scikit-learn. This way, 90% of the data is used for training and 10% for testing while maintaining the class distribution between walking and jumping samples:

```
1. X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42,
stratify=y)
```

We stored the resulting datasets in their respective groups within the HDF5 file, along with important metadata:

```
1. train_group.create_dataset('X', data=X_train)
2. train_group.create_dataset('y', data=y_train)
3. train_group.attrs['walking_samples'] = np.sum(y_train == 0)
4. train_group.attrs['jumping_samples'] = np.sum(y_train == 1)
5.
6. test_group.create_dataset('X', data=X_test)
7. test_group.create_dataset('y', data=y_test)
8. test_group.attrs['walking_samples'] = np.sum(y_test == 0)
9. test_group.attrs['jumping_samples'] = np.sum(y_test == 1)
10.
11. hf.attrs['description'] = 'Accelerometer data for walking vs jumping classification'
12. hf.attrs['sampling_rate'] = sampling_rate
13. hf.attrs['window_size'] = 5
```

```
14. hf.attrs['label_mapping'] = '0: Walking, 1: Jumping'
```

The final HDF5 file contained the complete dataset with the number of walking and jumping samples clearly documented as attributes. This data storage approach ensured data integrity throughout our analysis and facilitated collaboration among team members. The file structure also provides flexibility for future extensions.

3. Visualization

Visualizations were generated from the accelerometer data for walking and jumping collected from three team members: Rafael, Shehraj, and Victor.

3.1 Time Plots

Figures 2–7 show raw accelerometer data for each member, separately for walking and jumping:

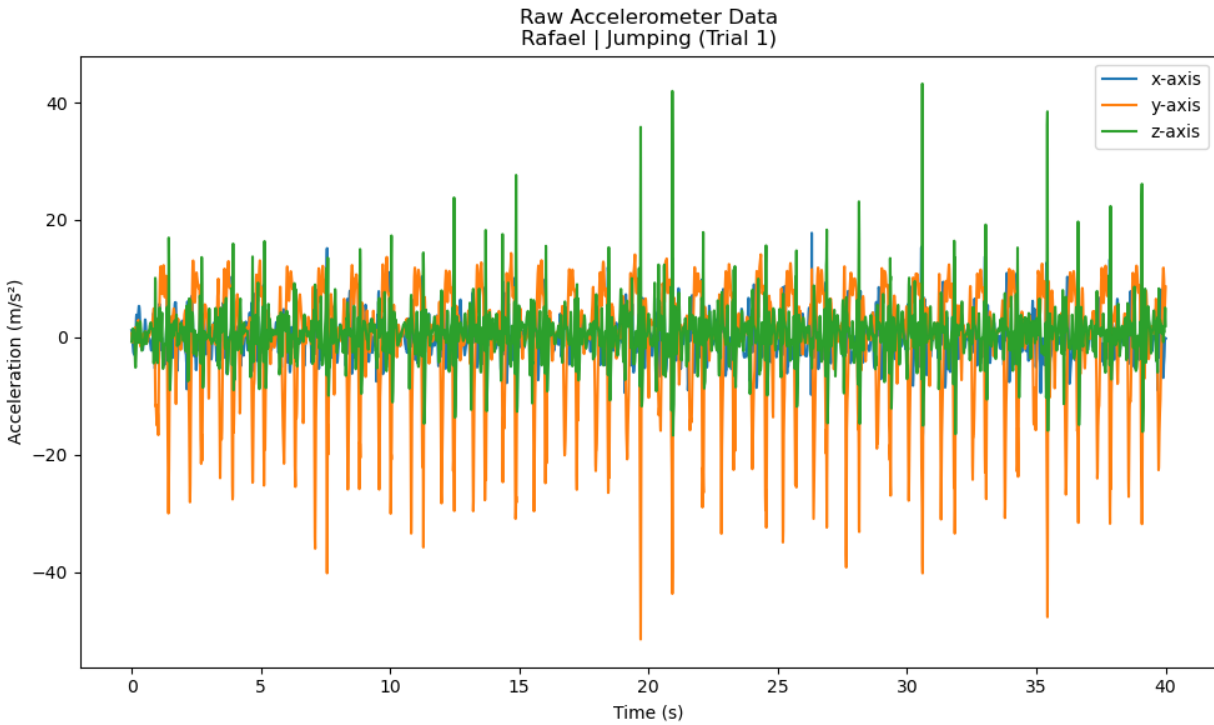


Figure 2: Raw accelerometer data for Rafael's jumping trial 1.

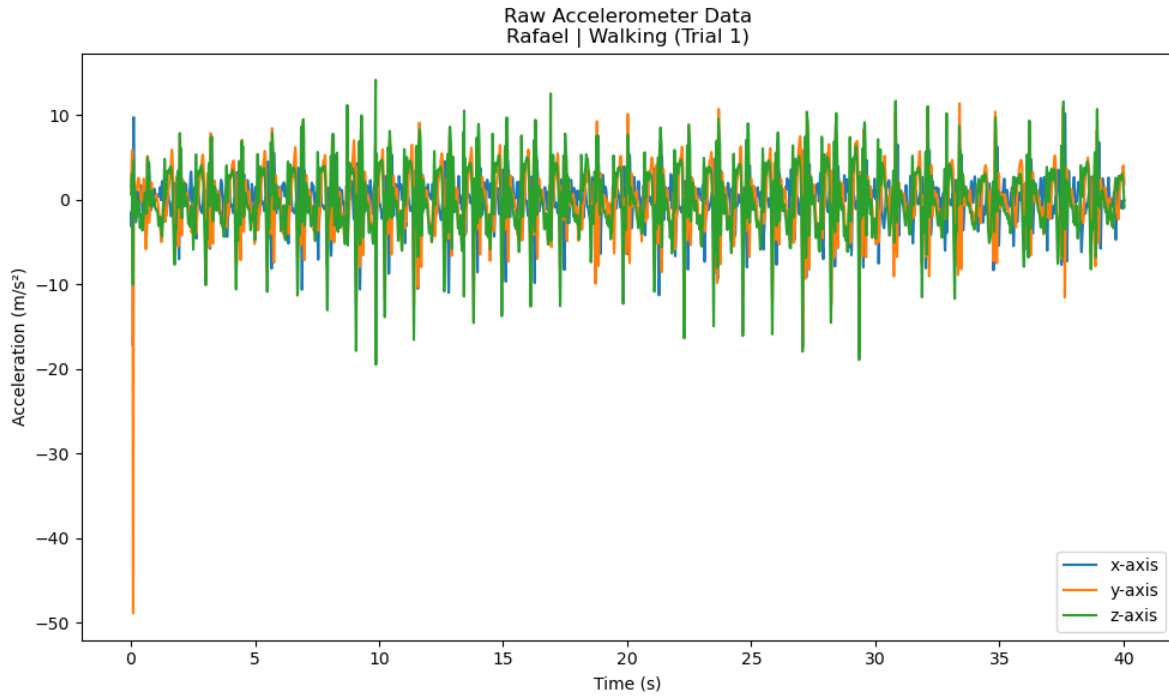


Figure 3: Raw accelerometer data for Rafael's walking trial 1.

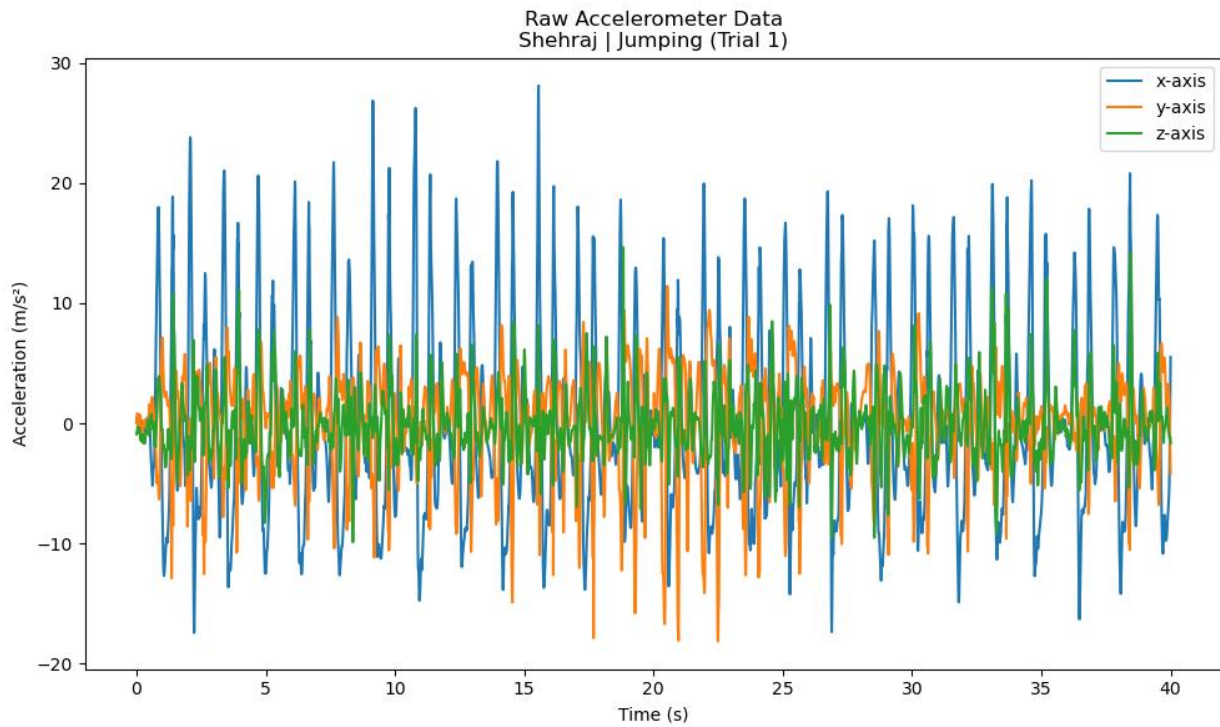


Figure 4: Raw accelerometer data for Shehraj's jumping trial 1.

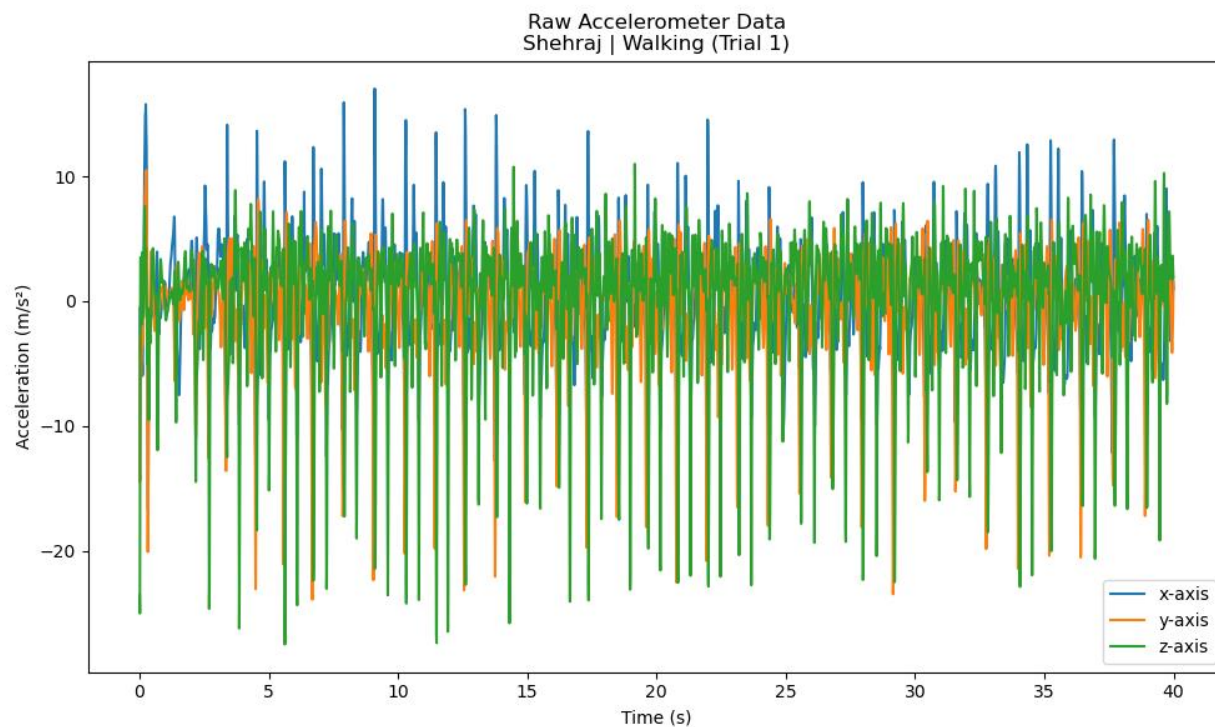


Figure 5: Raw accelerometer data for Shehraj's walking trial 1.

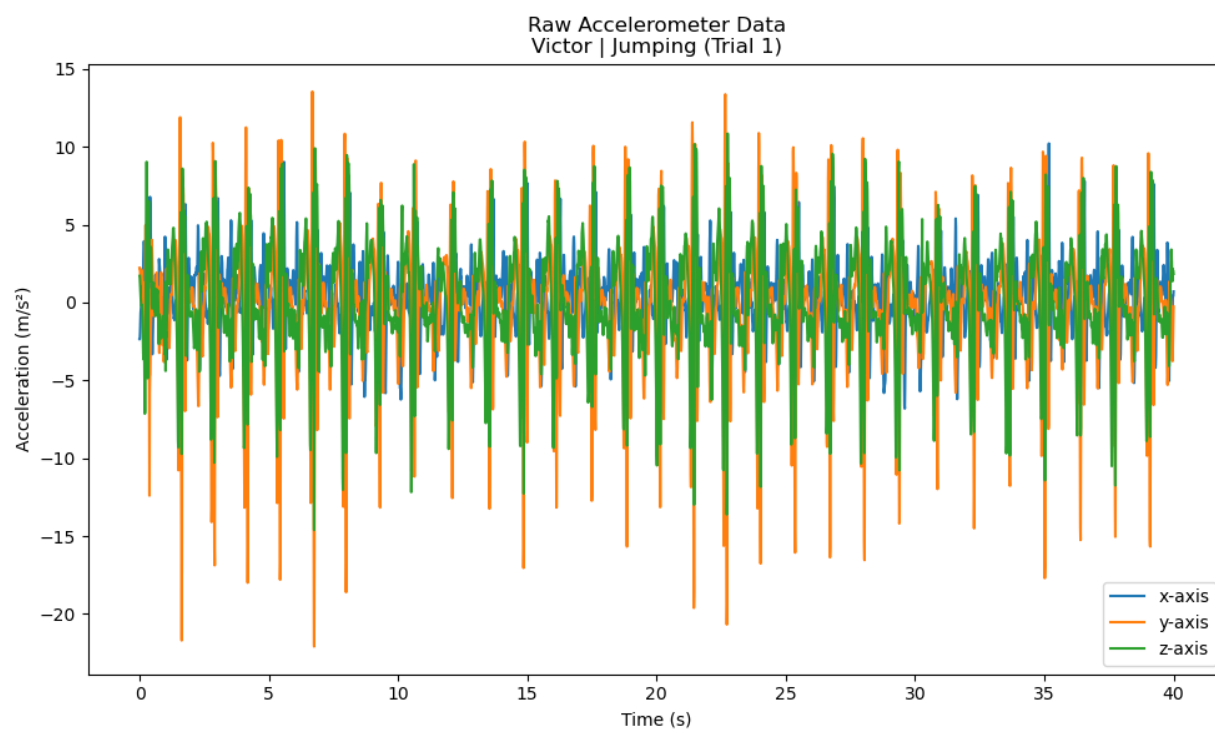


Figure 6: Raw accelerometer data for Victor's jumping trial 1.

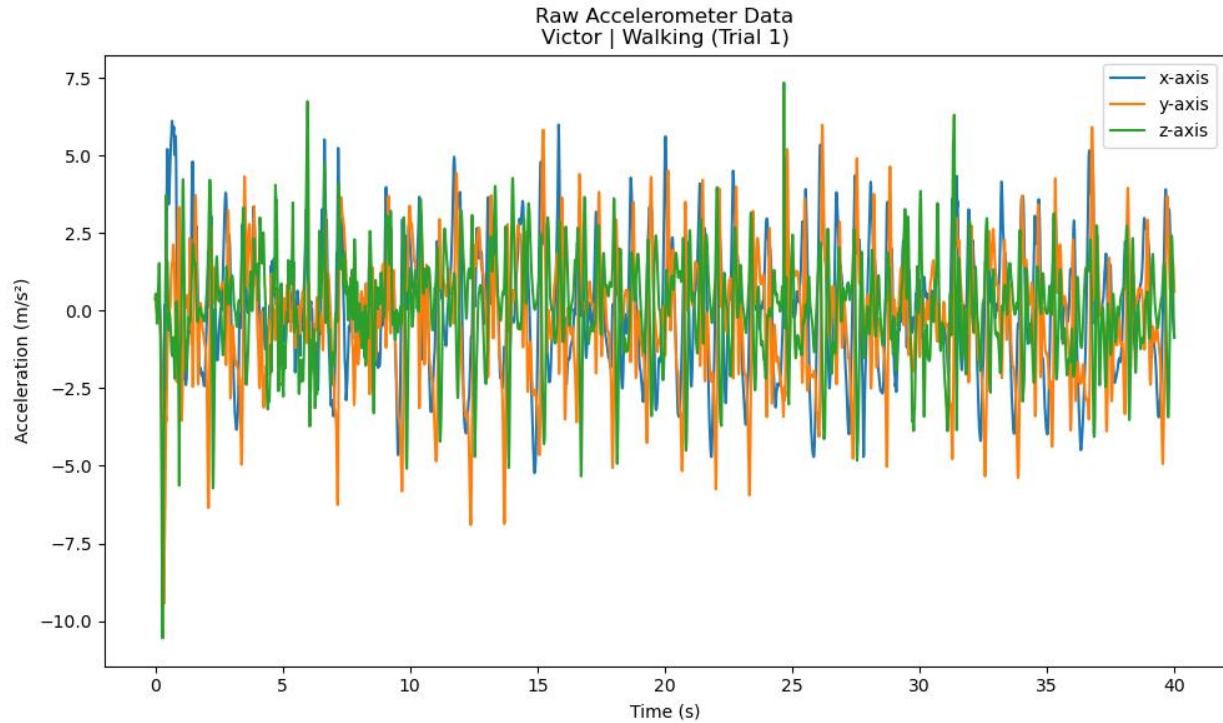


Figure 7: Raw accelerometer data for Victor's walking trial 1.

For Figure 2 and Figure 3 (Rafael), the jumping and walking trials show relatively consistent and distinguishable acceleration patterns, with jumping demonstrating notably higher and more sudden changes, particularly on the z-axis. For Figure 4 and Figure 5 (Shehraj) the walking data stands out due to significant fluctuations on both the z-axis and y-axis compared to the other group members. This difference suggests potential inconsistencies, possibly caused by different phone orientations or stride patterns. Finally, Figure 6 and Figure 7 (Victor) shows data that can clearly differentiate jumping from walking, as jumping exhibits higher changes in all axes. These plots revealed significant variation among individuals, highlighting potential inconsistencies in data collection methods, especially phone positions and type of movement.

3.2 Overlaid Plots

Figures 8–10 illustrate absolute acceleration comparisons between walking and jumping:

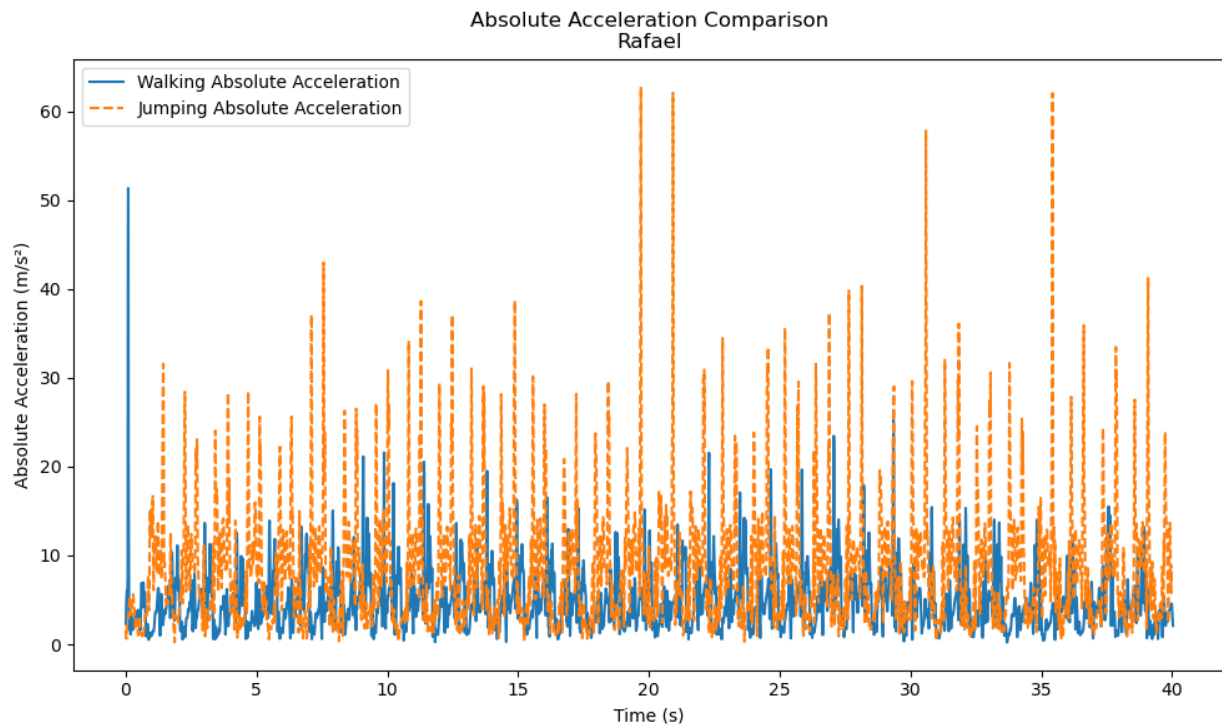


Figure 8: Rafael's absolute acceleration of walking and jumping, overlaid.

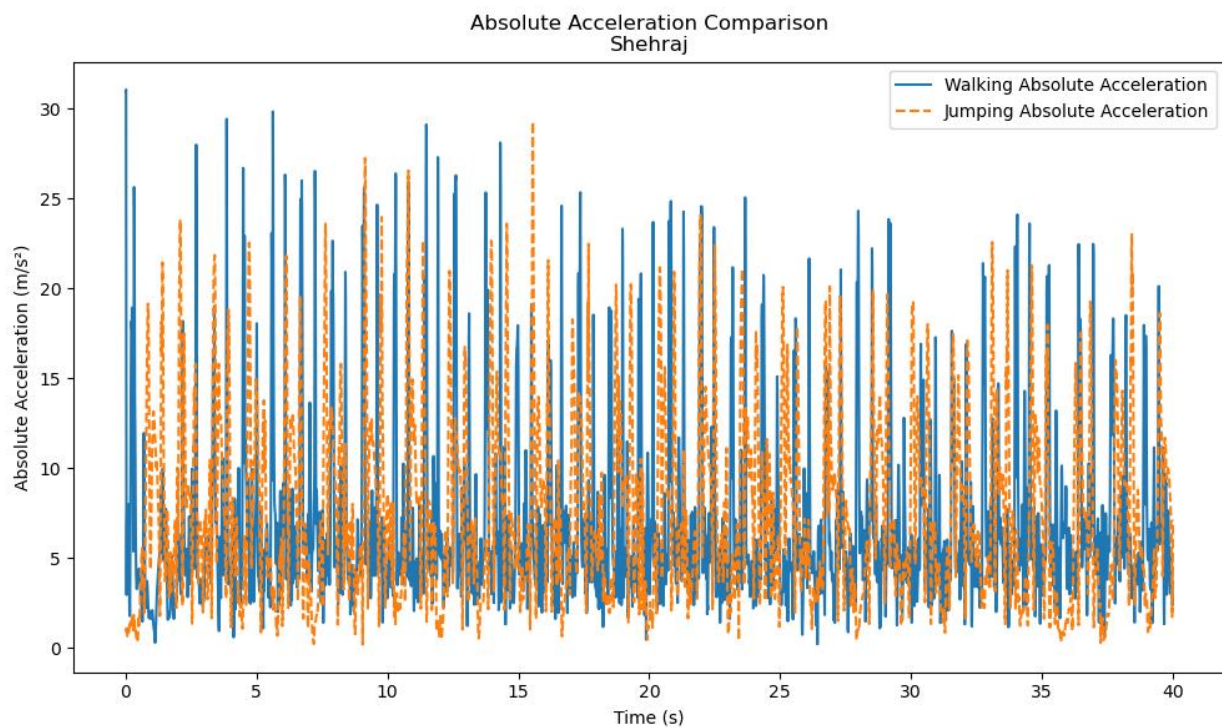


Figure 9: Shehraj's absolute acceleration of walking and jumping, overlaid.

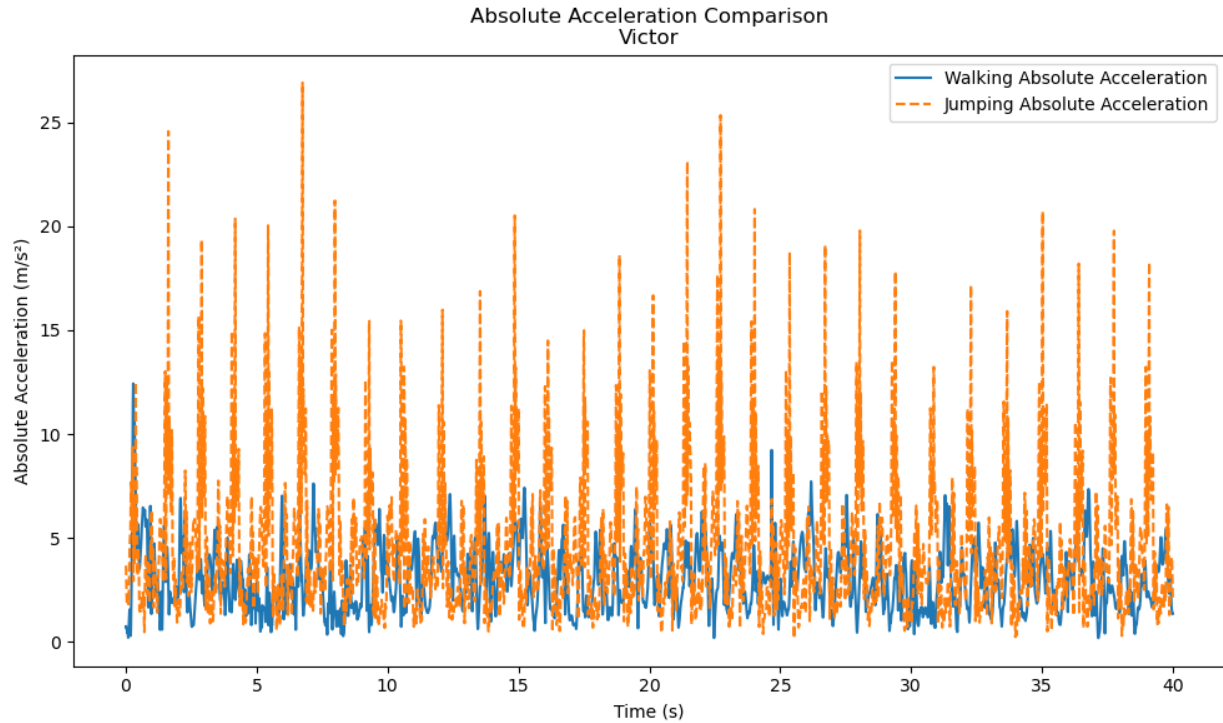


Figure 10: Victor's absolute acceleration of walking and jumping, overlaid.

Figure 8 (Rafael) & Figure 10 (Victor) both show distinct and clear separation between walking and jumping, with jumping having consistently higher acceleration magnitudes. Figure 9 (Shehraj) shows walking and jumping data that exhibit less clear distinction due to unusual patterns in walking data, showing potentially inconsistent data collection conditions or phone orientation. This information shows the importance of standardized data collection methods and shows where further improvements can be made in the data collection.

3.3 Metadata Visualizations

Figure 11 and Figure 12 show both the distribution of data between members and one of the features in the data.

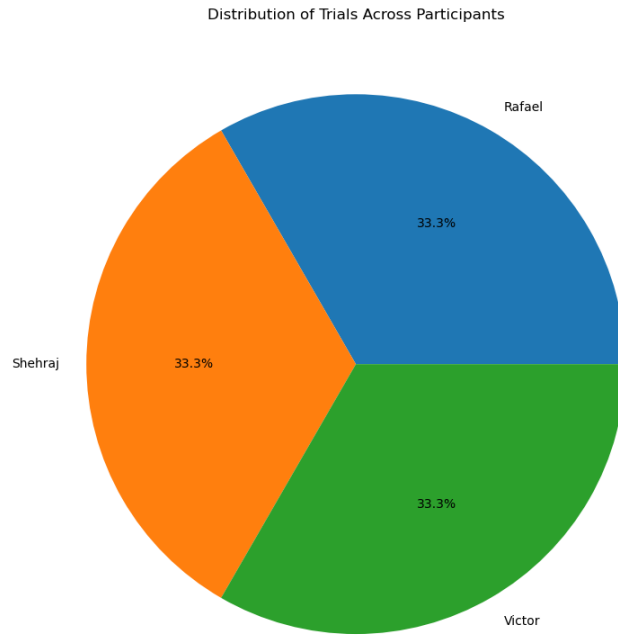


Figure 11: Distribution data used in training from each group member.

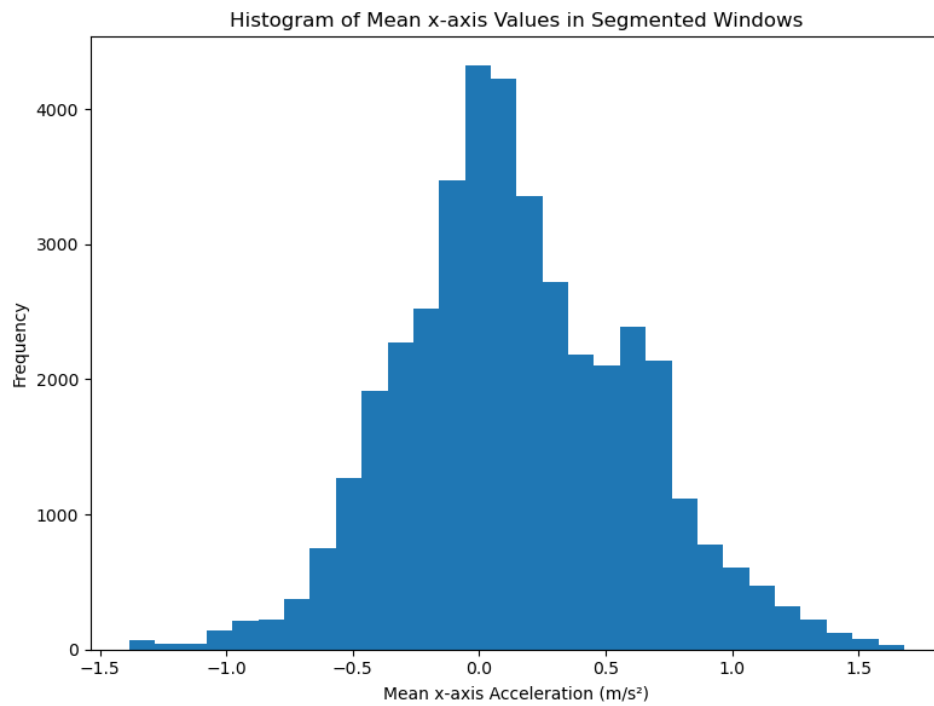


Figure 12: Histogram of the feature (mean) in the x-axis, and their frequencies.

Figure 11 confirms the equal data representation (33% each) from all team members, ensuring balanced data collection. Figure 12 shows the most common mean acceleration range is between -0.5 m/s^2 to 0.5 m/s^2 , suggesting walking and jumping patterns often have low average accelerations in the x-axis.

3.4 Insights Gained

By looking at the plots, the team saw that Shehraj's walking data highlighted issues such as inconsistent phone orientation or change in walking stride, potentially affecting the data. Also, there were clear differences in acceleration magnitudes between walking and jumping which confirmed the ability to differentiate between activities based on acceleration data. Then, the metadata confirmed balanced participation but also indicated an opportunity to increase trial diversity. If the team were to re-do the data collection, the team would conduct more trials per participant, ensuring diverse data points, standardize phone orientation and placement across all trials and members, and increase variations in movement scenarios to enhance data, providing a broader dataset. These steps would significantly improve the quality and consistency of future datasets, allowing for more accurate and reliable activity classification outcomes.

4. Preprocessing

Accelerometer data from smartphones inherently contains noise due to sensor limitations, hand movement, and the environment. Additionally, sampling rate inconsistencies can lead to missing values. Our preprocessing pipeline addressed these issues through noise reduction and missing value imputation.

4.1 Noise Reduction

We implemented a moving average filter to smooth the time-series data across all three accelerometer axes (x, y, z) and the magnitude. The moving average was applied using Pandas' `.rolling()` function with the following parameters:

```
1. processed_df['x'] = processed_df['x'].rolling(window=window_size, center=True).mean()
```

We chose the `center = True` parameter to ensure the window was centered on each data point rather than trailing behind it, which preserves the alignment of features like peaks and valleys in the signal.

4.2 Handling Missing Values

After applying the moving average filter, we addressed missing values through linear interpolation to estimate missing values based on surrounding data points:

```
1. processed_df = processed_df.interpolate(method='linear')
```

Also, backward fill for any remaining missing values at the beginning of the dataset:

```
1. processed_df = processed_df.fillna(method='bfill')
```

This combined approach minimizes data loss while maintaining signal integrity.

4.3 Choosing a Window Size

The window size for our moving average filter was determined through experimentation. We evaluated window sizes of 5, 15, and 30 data points and measured their impact on classification accuracy, as seen in Table 1.

Table 1: Window size and accuracy correlation, used for determining window size in processing.

Window Size	Accuracy
5	0.9628
15	0.9648
30	0.9628

While the accuracy differences were minimal, the team selected a window size of 15 as it provided the highest accuracy while maintaining a good balance between noise reduction and preservation of the signal's distinctive features. Smaller windows (e.g., 5) retained too much noise, while larger windows (e.g., 30) risked over-smoothing the signal and potentially eliminating important features that distinguish walking from jumping.

4.4 Effectiveness of Preprocessing

Figure 13 shows the effect of the preprocessing on the raw accelerometer data. The processed signal (in blue) shows significant reduction in random spikes compared to the raw signal (in red). Quantitatively, the variance of the signal was reduced by more than 50% after preprocessing, for both walking and jumping data, indicating substantial noise reduction while preserving the fundamental pattern differences between walking and jumping activities.

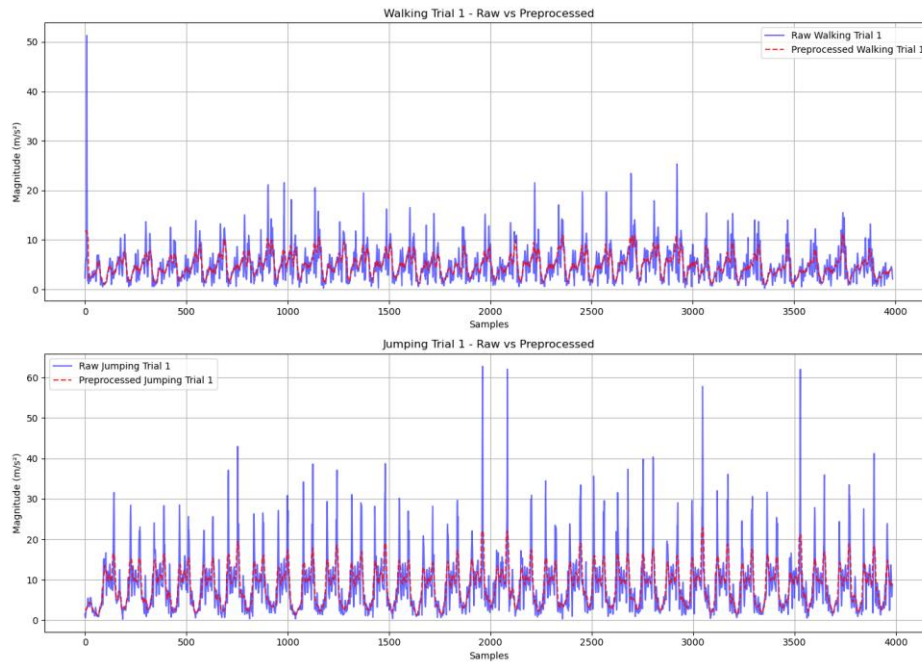


Figure 13: Plots showing the difference between the raw data and changes from preprocessing.

5. Feature Extraction & Normalization

For differentiating between walking and jumping, the team extracted a set of time-domain statistical features from each 5-second window of accelerometer data. The feature extraction approach was guided by previous research in human activity recognition using wearable sensors, which has established certain statistical measures as particularly effective for distinguishing between different motion patterns [1], [2].

5.1 Feature Selection

The following 12 features for each window of data: mean values for each axis (x, y, z) and magnitude, standard deviation for each axis and magnitude, and skewness for each axis and magnitude. The mean represents the average tendency of acceleration during the activity. Walking typically exhibits consistent mean values with smaller amplitude, while jumping shows greater variation and higher amplitude. This feature captures the orientation and overall intensity of movement. Then, the standard deviation quantifies the variability of the acceleration values. This feature captures the range of movements. Jumping activities generally produce higher standard deviations due to the rapid changes in acceleration during takeoff and landing. Finally, skewness describes the asymmetry of the probability distribution of acceleration values. Walking typically produces more symmetrical distributions, while jumping creates asymmetrical patterns due to the sudden acceleration changes.

5.2 Feature Extraction Process

The feature extraction process was implemented through a dedicated function that processes each window of accelerometer data:

```
1. def extract_features(window):
2.     features = {}
3.
4.     # Extract features for x-axis
5.     features['mean_x'] = np.mean(window[:, 0])
6.     features['std_x'] = np.std(window[:, 0])
7.     features['skew_x'] = skew(window[:, 0])
8.
9.     # Extract features for y-axis
10.    features['mean_y'] = np.mean(window[:, 1])
11.    features['std_y'] = np.std(window[:, 1])
12.    features['skew_y'] = skew(window[:, 1])
13.
14.    # Extract features for z-axis
15.    features['mean_z'] = np.mean(window[:, 2])
16.    features['std_z'] = np.std(window[:, 2])
17.    features['skew_z'] = skew(window[:, 2])
18.
19.    # Extract features for magnitude
20.    features['mean_magnitude'] = np.mean(window[:, 3])
21.    features['std_magnitude'] = np.std(window[:, 3])
22.    features['skew_magnitude'] = skew(window[:, 3])
23.
24.    return list(features.values())
```

This function calculates each statistical measure using NumPy for mean and standard deviation calculations, and SciPy for skewness computation. We then applied this function to each segmented window in our dataset:

```
1. X = np.array([extract_features(window) for window in all_windows])
2. y = np.array(all_labels)
```


5.3 Normalization Technique

Feature normalization is critical for our logistic regression model since features with larger scales could disproportionately influence the model's weights. We implemented standardization (z-score normalization) using scikit-learn's StandardScaler:

```
1. from sklearn.preprocessing import StandardScaler
2. from sklearn.pipeline import make_pipeline
3.
4. # Create pipeline with standardization and logistic regression
5. clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=10000))
```

Standardization transforms each feature to have zero mean and variance:

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

Where x is the original feature value, μ is the mean of the feature, and σ is the standard deviation of the feature. We chose standardization over min-max scaling because our features have different units and ranges, standardization is more robust to outliers compared to min-max scaling, and logistic regression performs better with standardized features, as the gradient descent optimization converges faster. The standardization parameters (mean and standard deviation) were estimated from the training data only and then applied to both training and test data to prevent data leakage.

6. Training & Testing the Classifier

The team selected logistic regression as our classification algorithm due to the project instructions, but also because of its efficiency with binary classification problems. Logistic regression provides probability estimates for class membership and works effectively with the numerical features the team extracted from the accelerometer data. This choice aligns with the project requirements while offering a good balance between simplicity and performance.

6.1 Training Pipeline

The model training pipeline incorporated data extraction, dimensionality transformation, and feature standardization. First, the team accessed the pre-segmented training and testing datasets from the HDF5 file structure, maintaining the separation between training and testing data to prevent information leakage:

```
1. with h5py.File(file_path, "r") as f:
2.     segmented_group = f["Segmented data"]
3.     x_train = segmented_group["Train"]["X"][:]
4.     y_train = segmented_group["Train"]["y"][:]
5.     x_test = segmented_group["Test"]["X"][:]
6.     y_test = segmented_group["Test"]["y"][:]
```

Then, each 5-second window containing multiple time points required reshaping to create a single feature per window:

```
1. x_train_flattened = x_train.reshape(x_train.shape[0], -1)
2. x_test_flattened = x_test.reshape(x_test.shape[0], -1)
```


This transformation converted the 3D data structure (windows $\times$ time points $\times$ features) into a 2D structure (windows $\times$ features) to be compatible with scikit-learn's implementation of logistic regression.

Finally, the team implemented standardization using StandardScaler to normalize feature distributions, ensuring that features with larger magnitudes did not disproportionately influence the model:

```
1. clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=10000))
```

This step was particularly important given the different scales of our extracted features (e.g., mean values vs. standard deviation values).

6.2 Model Training and Evaluation

The team configured the logistic regression model with an increased maximum iteration count of 10,000 to ensure convergence, given the complexity of our feature space:

```
1. l_reg = LogisticRegression(max_iter=10000)
```

We evaluated our model using multiple performance metrics to gain a comprehensive understanding of its capabilities. First, for the classification accuracy the model achieved an accuracy of 0.962, substantially exceeding both the chance level (50%) and the project requirement (80%). Then, the team obtained a recall score of 0.952, indicating the model's strong ability to correctly identify positive instances (jumping). Third, an ROC analysis. The receiver operating characteristic (ROC) curve analysis gave an area under the curve (AUC) of 0.9969, demonstrating the model's ability across different classification thresholds (seen in Figure 14).

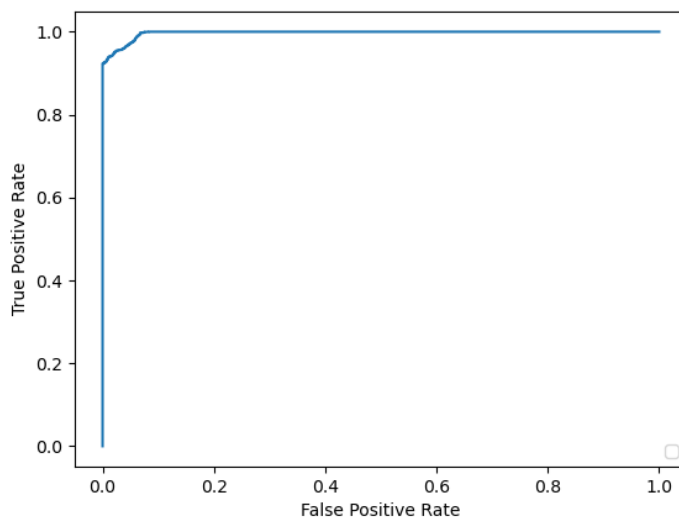


Figure 14: ROC curve plot showing relationship between true positive and false positive rate.

Figure 15 shows the confusion matrix for the test set predictions, revealing minimal misclassifications between walking and jumping activities. The matrix indicates that most errors occurred in classifying walking activities as walking (false negative) rather than the opposite.

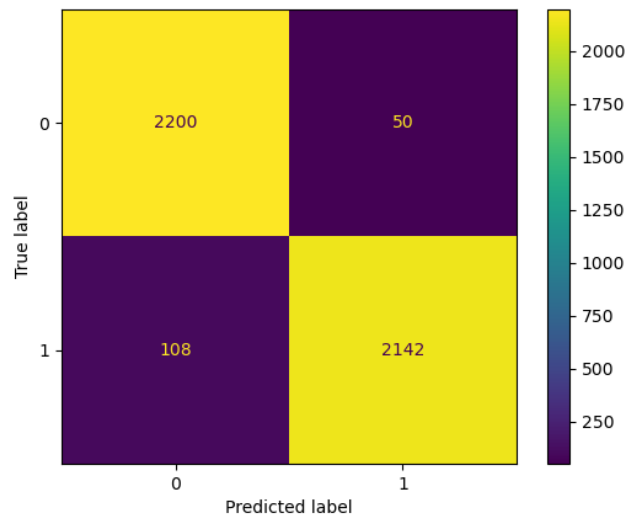


Figure 15: Confusion matrix displaying how many times a label was predicted vs what the label truly was.

To assess our model's learning efficiency and determine if additional training data would be beneficial, we performed a learning curve analysis with cross-validation (Figure 16). The learning curves illustrate how model performance varied with increasing training set sizes from 10% to 100% of our available data.

```

1. train_sizes, train_scores, val_scores = learning_curve(
2.     clf,
3.     x_train_flattened,
4.     y_train,
5.     cv=5,
6.     scoring='accuracy',
7.     train_sizes=np.linspace(0.1, 1.0, 10),
8.     n_jobs=-1
9. )

```

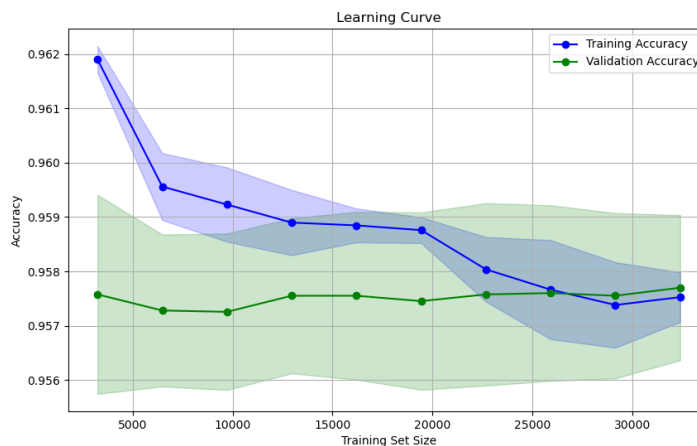


Figure 16: Training model learning curve as set size increases.

The learning curve analysis revealed that the training and validation scores converged with increasing training set size.

7. Model Development

The Graphical User Interface (GUI) was built using Customtkinter, with the intention of maximizing simplicity and ease of use. The GUI can be seen below in Figure 17. The interface is a 500x300 window and presents the user with three options. The buttons are designed to be pressed in order from top to bottom. First, a CSV file containing movement data (from an application such as Phyphox) is sent to the Classifier. Below is the process used to request a file from the user and open the file for next steps.

```
def open_file():  
    global csv_file_path  
    csv_file_path = filedialog.askopenfilename(filetypes=[("CSV Files", ".csv")])  
    if csv_file_path:  
        print("Selected file:", csv_file_path)
```

The second purpose of the GUI is to allow the user to process and display the data. This step processes the data by completing the previous steps, such as pre-processing, feature extraction. Code from these steps is reused to allow functionality of the interface. Once the program has determined if the user selected walking or jumping data, it will then display the result via a bar graph. This plot showcases how many windows were predicted as walking and jumping with great accuracy. Figure 18 shows an example of a plot output when processing walking data.

The final button on the Classifier is “Write to CSV” which uploads each window’s prediction result to a CSV file. The design choice to include three separate buttons was made because it makes the functionality of the GUI clear to the user, making it effective and user-friendly.

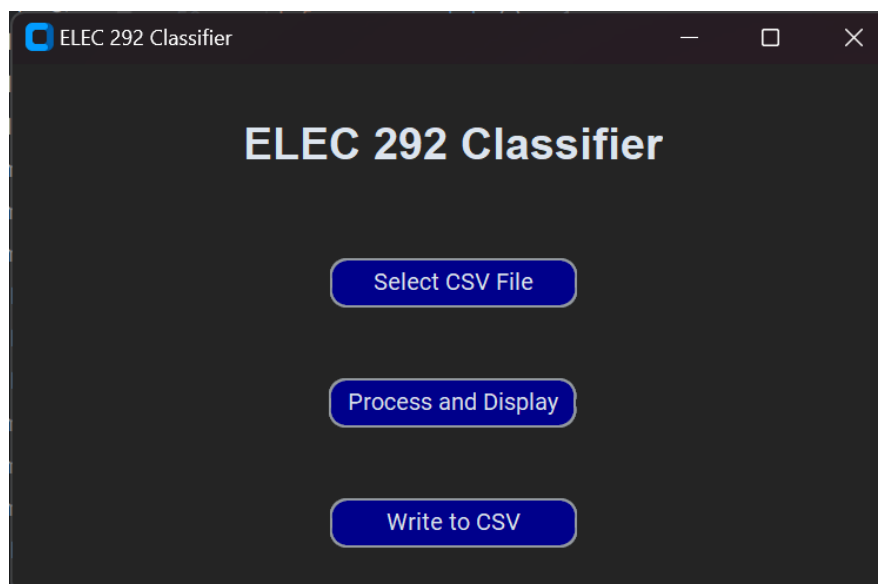


Figure 17: Graphical User Interface displaying different button functionalities.

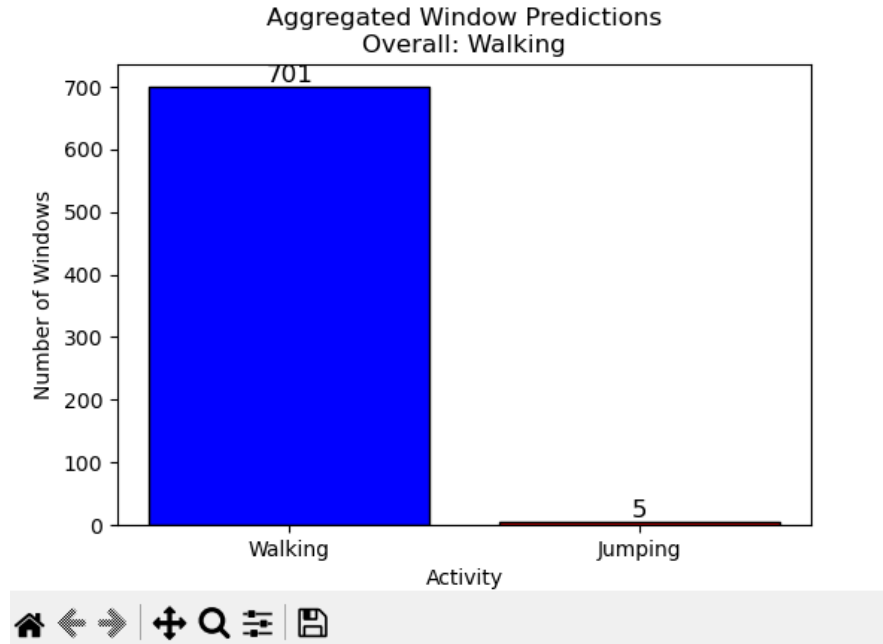


Figure 18: Result of Display Data button on GUI when provided with walking data, showcasing number of windows predicted as walking compared to jumping

Appendix

Table 2: Breakdown on work distribution in project and report.

Names	Sections Contributed
Shehraj	Data Collection, Graphical User Interface, Video Creation
Rafael	Data Storage, Preprocessing, Feature Extraction & Normalization, Training & Testing the Classifier.
Victor	Data Visualization, Preprocessing, and Feature Extraction

References

- [1] “Physical Human Activity Recognition Using Wearable Sensors - PMC.” Accessed: Apr. 03, 2025. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC4721778/?utm_
- [2] W. He, Y. Guo, C. Gao, and X. Li, “Recognition of human activities with wearable sensors,” *EURASIP Journal on Advances in Signal Processing*, vol. 2012, no. 1, p. 108, May 2012, doi: 10.1186/1687-6180-2012-108.